

Coding & Solution

Date	15 July 2026
Team ID	XXXXXX
Project Name	FinRelief AI – AI-Powered Debt Relief & Financial Recovery Platform
Maximum Marks	5 Marks

Coding & Solution

This section describes the implementation of FinRelief AI, an AI-powered platform that helps users manage debts, analyze their financial condition, and receive personalized debt settlement recommendations. The solution is developed using a modular architecture with React.js, FastAPI, SQLite, SQLAlchemy, and Google Gemini AI, ensuring maintainability, scalability, and secure data handling.

The application follows clean coding standards with reusable components, proper folder structure, REST APIs, authentication, and AI integration to provide users with an intelligent financial recovery system.

Instructions

1. Code Structure

- Well-organized frontend and backend folders.
- Reusable React components.
- Modular FastAPI backend.
- SQLAlchemy ORM for database operations.
- Secure JWT authentication.

2. Functional Requirements Implemented

- User Registration & Login
- Secure Authentication
- Loan Management
- Financial Health Analysis
- AI Debt Settlement Recommendations
- Dashboard with Loan Summary
- User Profile Management

3. Source Files Included

- React Frontend
- FastAPI Backend
- SQLite Database
- API Configuration
- Requirements.txt
- Package.json
- README.md

4. Repository

GitHub Repository: (Paste your GitHub repository URL here)

Solution Summary

Field	Details
Repository Link / URL	(Paste your GitHub Repository Link)

Programming Language(s)	Python, JavaScript, HTML5, CSS3
Framework(s) Used	React.js, FastAPI, SQLAlchemy, Tailwind CSS
Database	SQLite
Authentication	JWT Authentication
AI Integration	Google Gemini API
Key Features Implemented	User Registration, Login, Loan Management, Financial Dashboard, AI Financial Analysis, Debt Settlement Recommendation, Profile Management, REST APIs
Pending / Incomplete Features	Email Notifications, SMS Alerts, Bank API Integration, Multi-language Support, Cloud Deployment
Setup / Run Instructions	1. Install Node.js and Python 3.11+. 2. Install frontend dependencies using npm install. 3. Install backend dependencies using pip install -r requirements.txt. 4. Configure Gemini API key in environment variables. 5. Start backend using uvicorn main:app --reload. 6. Start frontend using npm run dev. 7. Open the application in your browser and register/login.

Technologies Used

- Frontend: React.js, HTML5, CSS3, Tailwind CSS
- Backend: FastAPI
- Programming Languages: Python, JavaScript
- Database: SQLite
- ORM: SQLAlchemy
- Authentication: JWT
- AI Model: Google Gemini API
- API Communication: Axios
- Version Control: Git & GitHub

Team Member

Jalli Ratna Mahitha